Техническое задание

Реализация клиента Avito Performance (Авито Реклама) на целевом языке (PHP / Go / Java / ...)

Эталонная Python-реализация: avito_performance.py. Цель — портировать функциональность с сохранением структуры выходных данных. Версия документа: 2026-06-10.

1. Цель проекта

Разработать библиотеку для работы с API Авито Реклама (Avito Performance). Библиотека загружает справочники (кампании, группы, креативы) и собирает дневную статистику (показы, клики, расход, видео-метрики) на трёх уровнях, плюс строит сводную таблицу-агрегат admin_audit. Выход каждой публичной функции — структурированный массив строк с фиксированным набором и порядком колонок.

2. Общая логика работы библиотеки

API синхронный, формат JSON — без submit/poll/download, без CSV/ZIP.

- Справочники (кампании / группы / креативы) — постраничный POST с телом {"filter": {}, "limit": 100, "page": N}; страницы собираются, пока не придёт пустой массив или page*limit >= total.
- Статистика — перебор сущностей: на каждую кампанию один POST .../stats с {"dateFrom", "dateTo"} (для групп/креативов — плюс массив их ID, сгруппированный по кампании из справочника). API возвращает data[] сразу за весь период (не более 100 дней) — разворачивать по дням локально, НЕ делать запрос на каждый день.
- Обогащение — константные и вычисляемые поля (разделы 3.7 и 4), затем колонки приводятся к фиксированному набору и порядку.

3. Ключевые принципы работы с Avito Performance (Авито Реклама)

3.1. Авторизация

- OAuth 2.0, grant_type=client_credentials: POST https://api.avito.ru/token.
- ВАЖНО: тело запроса — application/x-www-form-urlencoded (форма, НЕ JSON). Отправка JSON — типичная ошибка, токен не выдаётся.
- Ответ: {access_token, expires_in (86400 сек = 24 ч), token_type: Bearer}. Заголовок всех запросов к /ads/*: Authorization: Bearer <token>.
- Токен обновлять заранее — за 60 сек до истечения (TOKEN_REFRESH_LEEWAY_SEC = 60).
- Учётные данные: переменные окружения CLIENT_ID / CLIENT_SECRET; ID рекламного аккаунта — ACCOUNT_ID.
- Токен привязан к одному accountID: запрос к чужому аккаунту — 403. Единого агентского токена и метода «список кабинетов» нет. Один кабинет = отдельная тройка CLIENT_ID/CLIENT_SECRET/ACCOUNT_ID.

3.2. Базовый URL и эндпоинты

BASE_URL = https://api.avito.ru, все методы — под префиксом /ads/v1. Все запросы — POST. x-cost: справочники 1 балл/страница, campaigns stats 10, groups stats 5, creatives stats 1.

- POST /ads/v1/account/{accountID}/campaigns — справочник кампаний (постранично)
- POST /ads/v1/account/{accountID}/groups — справочник групп (постранично)
- POST /ads/v1/account/{accountID}/creatives — справочник креативов (постранично)
- POST /ads/v1/account/{accountID}/campaigns/{campaignID}/stats — статистика по кампании
- POST .../campaigns/{campaignID}/groups/stats — статистика по группам кампании
- POST .../campaigns/{campaignID}/creatives/stats — статистика по креативам кампании

3.3. Схема получения данных

Все методы синхронные — данные возвращаются в ответе сразу, без UUID и polling. Статистика отдаётся вложенным массивом data[] (элемент = день с timestamp); агрегат за период — totalData (в выгрузку не берём).

3.4. Лимиты API (критичны)

Лимит	Значение	Где применять
Запросов в минуту на аккаунт	500	При 429 — экспоненциальный backoff
API-баллы	x-cost у каждого метода	Остаток — в заголовке Api-Point-Balance
Период статистики	<= 100 дней	Валидировать dateFrom/dateTo на входе
Пагинация	limit 1–100 (дефолт 20)	Использовать limit=100, page >= 1

3.5. Обработка ошибок и rate-limit

- HTTP 429 — повтор с экспоненциальным backoff: пауза 1 сек, удвоение, до 5 повторов; уважать заголовок Retry-After.
- Единая схема ошибки: {"code": string, "message": string}.
- HTTP-коды: 400 (невалидное тело / период > 100 дней), 401 (нет/просрочен токен), 403 (токен не на этот accountID), 404, 429, 500, 503.
- Ошибка по одной кампании — warning в лог и продолжение цикла, не прерывать выгрузку.
- Кампании без активности в периоде возвращают пустой data[] — это не ошибка, строк не дают.

3.6. Подводные камни форматов данных

- Деньги — целые числа в рублях С НДС (spend, spendBonus, budget, price, balance); приходят как JSON integer.
- ВАЖНО: API отдаёт расходы целочисленным значением (целые рубли, без копеек). Из-за этого возможны незначительные расхождения со статистикой в личном кабинете Авито — там расходы отображаются с двумя знаками после запятой. Это особенность источника, а не ошибка выгрузки.
- costs_nds в выгрузке — десятичный тип (float), округление до 2 знаков сразу после чтения из API, до вычисления производных полей.
- Два формата дат в одном ответе (список кампаний): createdAt — ISO 8601 (2026-06-05T15:54:49Z); startDate/endDate/updatedAt — Go-time (2026-06-19 20:00:00 +0000 UTC, иногда с миллисекундами). У групп/креативов — ISO Z. Для ISO-Z НЕЛЬЗЯ конвертировать через локальную таймзону — время сдвинется.
- timestamp статистики приходит с офсетом +03:00 (МСК): дата строки = первые 10 символов.
- Неоднородный регистр ключей: advertiserId/contractId, но managerID/campaignID/advertiserID; в списке креативов groupID, в creatives/stats — groupId. Обязателен helper pick(row, "groupID", "groupId") — первый существующий ключ.
- Видео-метрики (videoViews25/50/75/100, vtr) — только у видео-кампаний; у textImage отсутствуют, в выгрузке NULL.
- filter: {} в справочниках возвращает сущности всех статусов. Поле haveCreative у групп в реальном ответе отсутствует.
- Метрики ctr, cpm, cpc, q25, q50, q75 из API НЕ выгружаются (исключены по требованию проекта). Охватных метрик (reach) в API нет вовсе.

3.7. Обогащение строк (соглашение проекта)

Константные и вычисляемые поля добавляются на стороне клиента до финальной сборки колонок. Значения констант — примеры, заменяются при интеграции.

- Для всех функций: account_id = 1, source_type_id = 9.
- Справочник кампаний: product_id = 1, product_name = "prod_test", camp_type = "camp_test", camp_category = "cat_test", owner_id = 1.
- Справочник групп: camp_type, camp_category, owner_id — join из справочника кампаний по campaign_id; group_type = "other", group_category = "other" — дефолт.

- Справочник креативов: id_key_group, owner_id — join из справочника групп по group_id.
- Расходы (ak = 0.5): costs_nds = spend (float, round 2); costs_without_nds = costs_nds / (1 + ставка НДС), ставка по году даты строки: год >= 2026 — 22% (делитель 1.22), раньше — 20% (делитель 1.20), per-row; costs_nds_ak = costs_nds * 1.5; costs_without_nds_ak = costs_without_nds * 1.5.
- Ключи: id_key_camp = "1_" + campaign_id (все функции); id_key_group = id_key_camp + "_" + group_id (справочники групп и креативов); id_key_ad = id_key_group + "_" + creative_id (только ad-level).

4. Публичные функции (7 шт.)

ВАЖНО: максимальный период статистики — 100 дней. Все stat-функции (4.4-4.7) принимают date_from / date_to (YYYY-MM-DD); разница между ними не должна превышать 100 дней (PERIOD_MAX_DAYS = 100) — ограничение API: при превышении сервер возвращает 400. Валидировать период на входе; за более длинные интервалы — несколько последовательных вызовов с разбиением периода на куски не более 100 дней.

4.1. get_campaign_dict() — справочник кампаний

POST /ads/v1/account/{accountID}/campaigns, постранично. Дедупликация по campaign_id (выживает первая строка); строки без id отбрасываются. 14 колонок:

Поле	Тип	Источник (ключ API)
campaign_id	integer	id
campaign_name	string	name
campaign_type	string	campaignType (textImage/HTML/video)
advertiser_id	integer	advertiserId
contract_id	integer	contractId
manager_id	integer	managerID (заглавные ID)
account_id	integer	константа 1
source_type_id	integer	константа 9
product_id	integer	константа 1
product_name	string	константа "prod_test"
camp_type	string	константа "camp_test"
camp_category	string	константа "cat_test"
id_key_camp	string	вычисляется: "1_" + campaign_id
owner_id	integer	константа 1

4.2. get_group_dict() — справочник групп

POST /ads/v1/account/{accountID}/groups, постранично. 17 колонок:

Поле	Тип	Источник (ключ API)
group_id	integer	id
group_name	string	name
campaign_id	integer	campaignID
status	string	status
payment_model	string	paymentModel (CPM/CPC)
format	string	format (media/native/...)
advertiser_id	integer	advertiserID
manager_id	integer	managerID
account_id	integer	константа 1
source_type_id	integer	константа 9
camp_type	string	join из справочника кампаний по campaign_id
camp_category	string	join из справочника кампаний
group_type	string	дефолт "other"
group_category	string	дефолт "other"
id_key_camp	string	"1_" + campaign_id
id_key_group	string	id_key_camp + "_" + group_id
owner_id	integer	join из справочника кампаний

4.3. get_creative_dict() — справочник креативов (объявлений)

POST /ads/v1/account/{accountID}/creatives, постранично. 22 колонки: creative_id (id), creative_name (name), group_id (groupID), campaign_id (campaignID), status, payment_model (paymentModel), campaign_type (campaignType), title, description, button_text (buttonText), link, origin_url (originUrl), preview_url (previewUrl), landing_id (landingID), advertiser_id (advertiserID), manager_id (managerID) + обогащение: account_id, source_type_id, id_key_camp, id_key_group, id_key_ad, owner_id.

- id_key_group, owner_id — join из справочника групп по group_id; id_key_ad = id_key_group + "_" + creative_id.
- buttonText, originUrl, landingID в реальном ответе отсутствуют — NULL; title/description могут быть пустыми.

4.4. get_campaigns_daily_stat(date_from, date_to) — статистика по кампаниям

Перебор кампаний из справочника: на каждую POST .../campaigns/{id}/stats с {"dateFrom", "dateTo"}; разворачивается campaign.data[]. 20 колонок: date (timestamp[:10]), campaign_id, payment_model, campaign_type, метрики impressions (из API views), clicks, costs_nds (из spend), spend_bonus (spendBonus), video_views_25/50/75/100 (videoViews*), vtr, производные costs_without_nds, ak, costs_nds_ak, costs_without_nds_ak, account_id, source_type_id, id_key_camp.

ВАЖНО: поля названия (campaign_name) в статистике нет — имена джойнить из справочника по campaign_id. Метрика показов называется impressions (в API — views).

4.5. get_groups_daily_stat(date_from, date_to) — статистика по группам

ID групп из справочника группируются по campaign_id; на кампанию один POST .../groups/stats с {"groupIDs": [...], "dateFrom", "dateTo"}; разворачивается groups[].data[]. 19 колонок: как 4.4, но вместо payment_model/campaign_type — group_id (название группы не выгружается).

4.6. get_creatives_daily_stat(date_from, date_to) — статистика по креативам

ID креативов группируются по campaign_id; на кампанию один POST .../creatives/stats с {"creativeIDs": [...], "dateFrom", "dateTo"}; разворачивается creatives[].data[]. 21 колонка: date, campaign_id, group_id (из creatives[].groupId — строчная d!), creative_id + метрики и производные + account_id, source_type_id, id_key_camp, id_key_ad = id_key_camp + "_" + group_id + "_" + creative_id.

4.7. get_admin_audit(date_from, date_to) — сводный аудит (таблица admin_audit)

Собственного эндпоинта нет — агрегат поверх 4.4: к строкам статистики кампаний джойнится owner_id из справочника кампаний, затем метрики суммируются по ключу date x account_id x source_type_id x owner_id. 9 колонок:

Поле	Тип	Источник
date	string	ключ группировки
account_id	integer	константа 1 (ключ группировки)
source_type_id	integer	константа 9 (ключ группировки)
owner_id	integer	join из справочника кампаний (ключ группировки)
impressions	integer	сумма
clicks	integer	сумма
costs_nds	float	сумма, округление до 2 знаков
costs_without_nds	float	сумма
chef_flag	integer	константа 1 (дефолт)

5. Алгоритм сбора статистики (общий шаблон)

```
1. POST /ads/v1/account/{accountID}/campaigns (постранично) -> список кампаний.
2. Для групп/креативов: POST .../groups | .../creatives (постранично) ->
   ID сущностей, сгруппированные по campaign_id.
3. Для каждой кампании:
     a. POST .../campaigns/{id}/stats (или groups/stats | creatives/stats
        с массивом ID) с {"dateFrom": ..., "dateTo": ...}.
```

```
     b. Развернуть data[] -> по строке на (сущность x день);
        date = timestamp[:10]; пустой data[] -> пропустить.
     c. При ошибке - лог + продолжить со следующей кампанией.
4. Обогащение: costs_nds -> float.round(2); costs_without_nds по ставке НДС
   года строки; ak / costs_*_ak; константы; составные ключи.
5. Привести к фиксированному набору и порядку колонок.
```

6. Примеры запросов и ответов API (реальные)

6.1. Авторизация

```
POST https://api.avito.ru/token
Content-Type: application/x-www-form-urlencoded

client_id=<CLIENT_ID>&client_secret=<CLIENT_SECRET>&grant_type=client_credentials

Ответ:
{ "access_token": "eyJhbGc...", "expires_in": 86400, "token_type": "Bearer" }
```

6.2. Справочник кампаний

```
POST https://api.avito.ru/ads/v1/account/{accountID}/campaigns
Authorization: Bearer <token>
Content-Type: application/json

{ "filter": {}, "limit": 100, "page": 1 }

Ответ (фрагмент):
{
  "campaigns": [
    {
      "id": 297840407,
      "name": "5ka_rf-KomboZvezdy_06_2026_med",
      "status": "active",
      "budget": 1390800,
      "paymentModel": "CPM",
      "campaignType": "textImage",
      "startDate": "2026-06-05 15:47:00.692 +0000 UTC",
      "endDate": "2026-06-19 20:00:00 +0000 UTC",
      "advertiserId": 574505283,
      "contractId": 786670188,
      "managerID": 313033278,
      "createdAt": "2026-06-05T15:54:49Z",
      "updatedAt": "2026-06-19 20:00:00 +0000 UTC"
    }
  ],
  "total": 17
}
```

6.3. Статистика по кампании

```
POST https://api.avito.ru/ads/v1/account/{accountID}/campaigns/272290369/stats
Authorization: Bearer <token>
Content-Type: application/json

{ "dateFrom": "2026-06-02", "dateTo": "2026-06-02" }

Ответ (фрагмент; spend - JSON integer, рубли с НДС):
{
  "campaign": {
    "id": 272290369,
    "paymentModel": "CPC",
    "campaignType": "textImage",
    "data": [
```

```
      { "timestamp": "2026-06-02T00:00:00+03:00", "views": 26202, "clicks": 36,
        "spend": 151, "spendBonus": 0,
        "ctr": 0.00137, "cpc": 420.75, "cpm": 578.08 }
    ],
    "totalData": { "views": 26202, "clicks": 36, "spend": 151 }
  }
}

ctr/cpc/cpm и totalData в выгрузку не берутся.
```

6.4. Статистика по группам / креативам

```
POST .../campaigns/{campaignID}/groups/stats
{ "groupIDs": [548487753547], "dateFrom": "2026-06-02", "dateTo": "2026-06-02" }
-> { "groups": [ { "id": 548487753547, "data": [ {...} ] } ] }

POST .../campaigns/{campaignID}/creatives/stats
{ "creativeIDs": [601237520391766], "dateFrom": "2026-06-02", "dateTo": "2026-06-02" }
-> { "creatives": [ { "id": 601237520391766, "groupId": 548487753547,
                      "data": [ {...} ] } ] }

ВАЖНО: в creatives/stats поле группы - groupId (строчная d),
в списке креативов - groupID.
```

7. Примеры таблиц на выходе (реальные строки)

7.1. get_campaign_dict — справочник кампаний

campaign_id	campaign_name	campaign_type	advertiser_id	contract_id	manager_id
509702671	5ka_rf-barni_05_2026_nat	textImage	574505283	786670188	313033278
297840407	5ka_rf-KomboZvezdy_06_2026_med	textImage	574505283	786670188	313033278

Константные/вычисляемые поля (для тех же 2 строк в том же порядке):

account_id	source_type_id	product_id	product_name	camp_type	camp_category	id_key_camp	owner_id
1	9	1	prod_test	camp_test	cat_test	1_509702671	1
1	9	1	prod_test	camp_test	cat_test	1_297840407	1

7.2. get_group_dict — справочник групп

group_id	group_name	campaign_id	status	payment_model	format
148335493810	Группа – 1	297840407	active	CPM	media
638420779651	Группа – 1	563334516	active	CPC	native

Обогащение (для тех же 2 строк; camp_type/camp_category/owner_id — join из кампаний):

camp_type	camp_category	group_type	group_category	id_key_camp	id_key_group	owner_id
camp_test	cat_test	other	other	1_297840407	1_297840407_148335493810	1
camp_test	cat_test	other	other	1_563334516	1_563334516_638420779651	1

7.3. get_creative_dict — справочник креативов

creative_id	creative_name	group_id	campaign_id	status
213211925758022	Ножи	203449948197	322501250	draft
786710393921224	twix	148335493810	297840407	active

Ключи (для тех же 2 строк; id_key_group/owner_id — join из справочника групп):

id_key_camp	id_key_group	id_key_ad	owner_id
1_322501250	1_322501250_203449948197	1_322501250_203449948197_213211925758022	1
1_297840407	1_297840407_148335493810	1_297840407_148335493810_786710393921224	1

7.4. get_campaigns_daily_stat — статистика по кампаниям

date	campaign_id	payment_model	impressions	clicks	costs_nds	costs_without_nds
2026-06-02	563334516	CPC	1048189	1954	13511.0	11074.59
2026-06-02	534075762	CPM	109316	196	23393.0	19174.59

Производные и константы (для тех же 2 строк): ak=0.5; costs_nds_ak: 20266.5 / 35089.5; costs_without_nds_ak: 16611.89 / 28761.89; account_id=1; source_type_id=9; id_key_camp: 1_563334516 / 1_534075762. Поля spend_bonus, video_views_25/50/75/100, vtr присутствуют в выгрузке (у textImage-кампаний видео-метрики NULL).

7.5. get_groups_daily_stat / get_creatives_daily_stat

date	campaign_id	group_id	creative_id	impressions	clicks	costs_nds
2026-05-30	272290369	548487753547	— (группы)	1484299	1777	31991.0
2026-05-30	272290369	548487753547	601237520391766	1484299	1777	31991.0

id_key_ad для строки креатива: 1_272290369_548487753547_601237520391766.

7.6. get_admin_audit — сводный аудит

date	account_id	source_type_id	owner_id	impressions	clicks	costs_nds	costs_without_nds	chef_flag
2026-05-31	1	9	1	1301169	1779	22938.0	18801.64	1
2026-06-01	1	9	1	2271039	2841	149688.0	122695.08	1
2026-06-02	1	9	1	3175332	5069	206898.0	169588.52	1

8. Рекомендации по реализации

- HTTP-клиент с retry-middleware для 429 (backoff 1-2-4-8-16 сек, макс. 5 повторов, уважать Retry-After); таймаут запроса 30 сек.
- Кэширование токена: token + expiresAt; ensureToken() перед каждым запросом, обновление за 60 сек до истечения.
- pick()-helper: первый существующий ключ из вариантов (pick(row, "groupID", "groupId")) — обязателен из-за неоднородного регистра ключей API.
- Нормализация дат: ISO-Z — без сдвига таймзоны; Go-time (2026-06-19 20:00:00 +0000 UTC) — парсить отдельно; день статистики = timestamp[:10].
- Деньги: integer на входе -> десятичный тип, округление до 2 знаков (costs_nds); ставка НДС по году даты строки (>= 2026 -> 1.22, иначе 1.20).
- Порядок вызова справочников: кампании -> группы (join полей кампаний) -> креативы (join полей групп). admin_audit использует статистику кампаний + справочник кампаний.
- Логирование: пропуск сущности при ошибке — уровень warning, без остановки процесса.
- Константы: PAGE_LIMIT=100, PERIOD_MAX_DAYS=100, HTTP_TIMEOUT_SEC=30, TOKEN_TTL_SEC=86400, TOKEN_REFRESH_LEEWAY_SEC=60, RATE_LIMIT_BASE_SEC=1, RATE_LIMIT_RETRY_MAX=5.
- Тесты: моки HTTP-клиента на все 7 публичных функций + retry на 429 + пустые ответы (пустой результат с правильными колонками) + смешанные форматы дат + дедупликация ID.

9. Критерии приёмки

- Все 7 публичных функций возвращают данные с теми же колонками и в том же порядке, что в Python-эталоне avito_performance.py (разделы 4.1-4.7).
- Токен запрашивается формой (x-www-form-urlencoded) и автоматически переподписывается при многочасовом цикле.
- Корректная работа на реальных учётных данных: не менее 15 кампаний, период более 7 дней, без ошибок 429 (backoff отрабатывает).
- costs_nds — десятичный тип с округлением до 2 знаков; costs_without_nds — по ставке НДС года строки (1.22 / 1.20).
- Строки разворачиваются по дням локально из data[] (1 запрос на сущность за период, НЕ по запросу на день).
- Составные ключи id_key_camp / id_key_group / id_key_ad — по формулам раздела 3.7.

- При сбое любой кампании процесс продолжает работать, ошибка попадает в лог.
- README с примерами вызова + .env.example (CLIENT_ID, CLIENT_SECRET, ACCOUNT_ID).